Generating a PWM Waveform

LAB # 07

Spring 2024

CSE-307L Microprocessor Based System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

"On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work."

Submitted to:

Dr. Asif Ali Khan

Date:

28^{th} April 2024

Department of Computer Systems Engineering

University of Engineering and Technology, Peshawar

Pulse Width Modulation:

PWM is an abbreviation of Pulse Width Modulation. Pulse width modulation is basically a square wave with a varying high and low time.

Basic Terminologies:

There are various terms associated with PWM:

- **On-Time:** Duration of time signal is high.
- **Off-Time:** Duration of time signal is low.
- **Period:** It is represented as the sum of on-time and off-time of PWM signal
- **Duty cycle:** It is represented as percentage of time signal remains on during the period of the PWM signal.

Applications:

- It is used in those engineering projects where you want an analog output.
- For example, if you want to control the speed of your DC motor then you need PWM pulse.
- Using PWM signal you can move your motor at any speed from 0 to its max speed.
- Similarly suppose you want to dim your LED light, again you will use PWM pulse.

Task 1:

Generate a signal of 500Hz with 40% duty cycle.

Calculations:

Frequency of signal, f = 500Hz

Time Period = 1/f = 1/500 = 0.002s = 2ms

On-Time = 1ms

Off-Time = 1ms

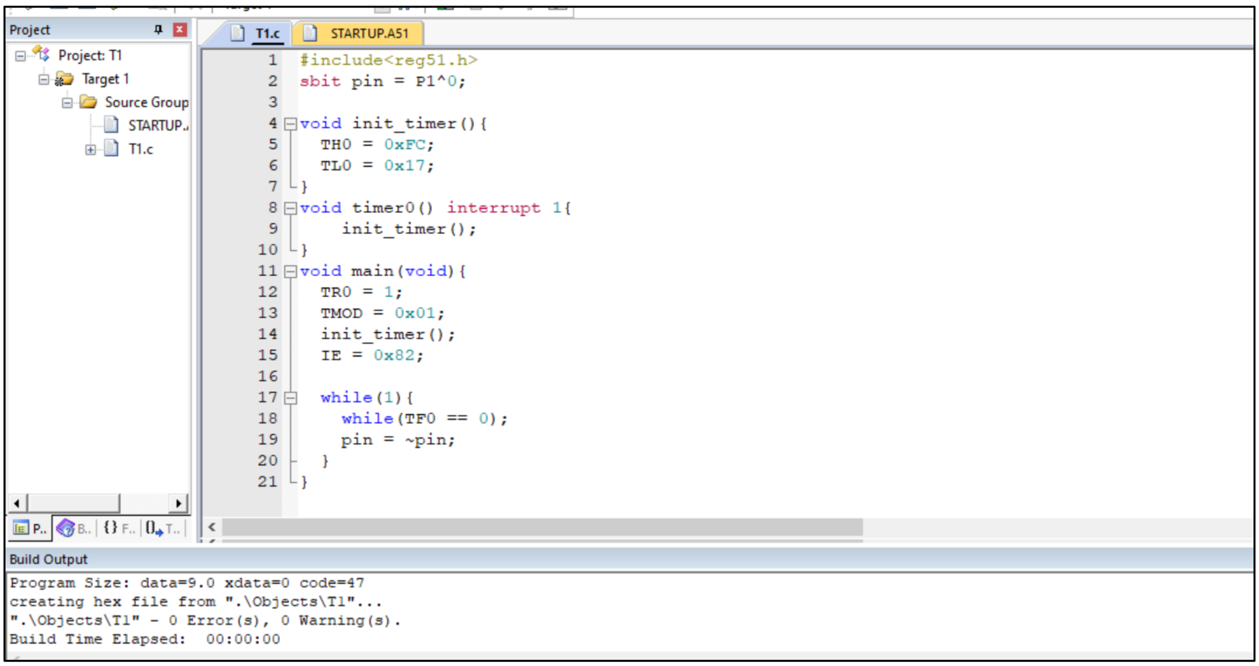
So, we need to create a time delay of 1ms in our program.

For 1ms delay, subtract 1000_{dec} from $FFFF_{hex}$

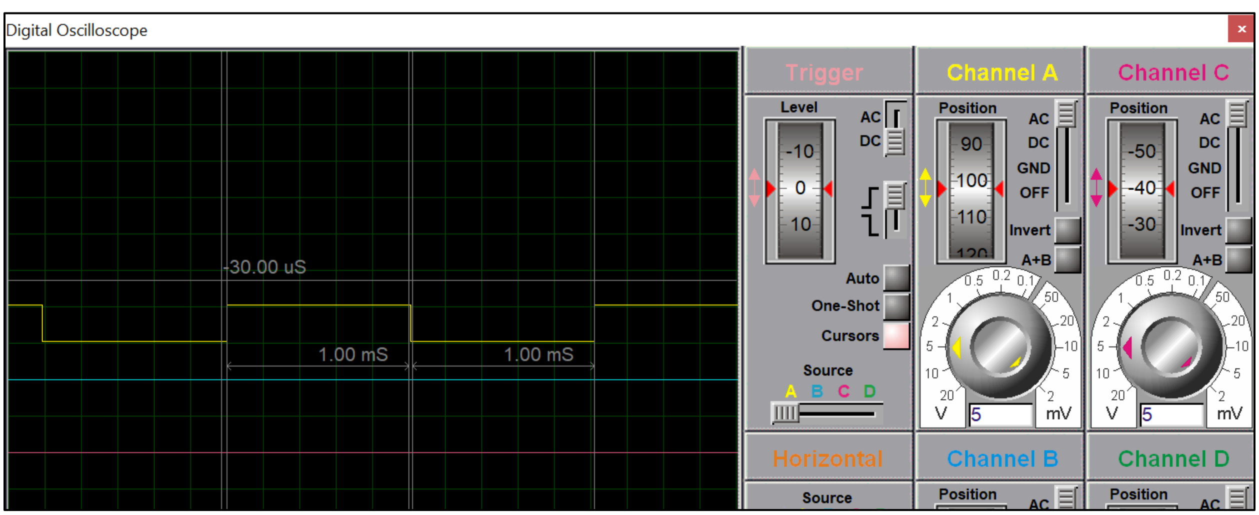
Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex}$ - 1000_{dec} = **$FC17_{hex}$**

Code:



Proteus Simulation



Task 2:

Generate a signal of 600Hz with 60% duty cycle on P1.3

- Hint: use timer

Calculation:

Frequency of signal, f = 600Hz

Time Period, T = 1/f = 1/600 = 0.00166s = 1.66ms

On-Time = 60% of 1.66ms = 0.996ms

Off-Time = 1.66 – **(On-Time)** = 0.664ms

So, we need to create toe time delays of 1.66ms and 0.996ms in our program.

For 0.996ms:

Subtract 996_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex} - 996_{dec} = FC1B_{\mathbf{hex}}$

For 0.664ms:

Subtract 664_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

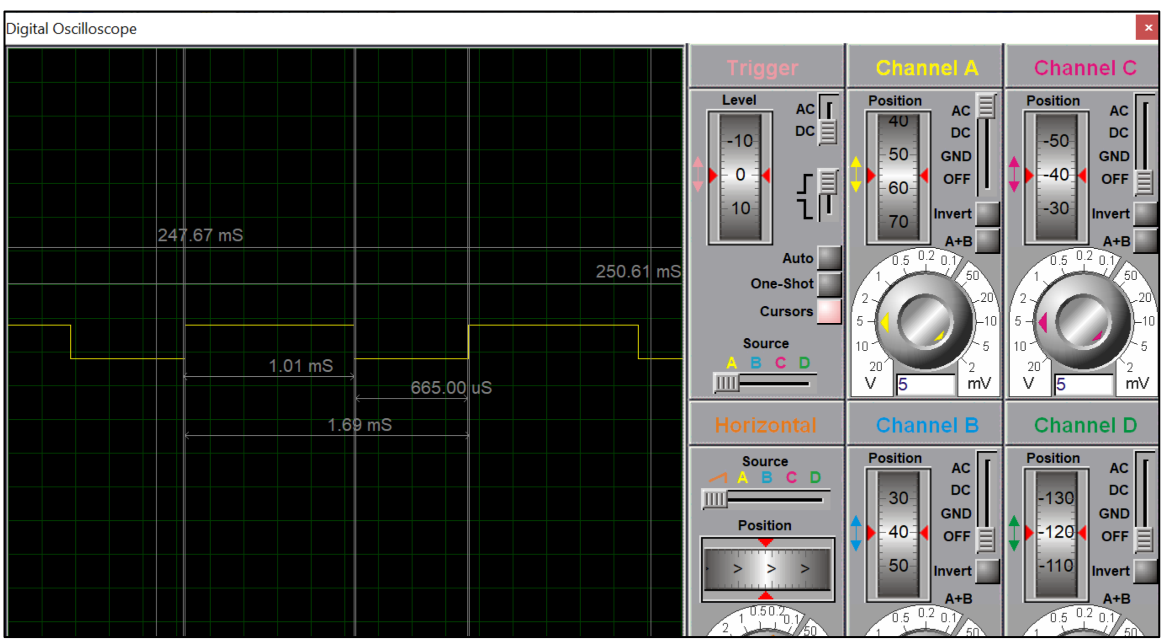
$FFFF_{hex} - 664_{dec} = FD67_{\mathbf{hex}}$

Code:

T2.c

```c
#include<reg51.h>
sbit pin = P1^0;

void init_timer(){
  if(pin){
    TH0 = 0xFD;
    TL0 = 0x67;
  }
  else{
    TH0 = 0xFC;
    TL0 = 0x1B;
  }
}

void timer0() interrupt 1{
    init_timer();
}

void main(void){
  TR0 = 1;
  TMOD = 0x01;
  init_timer();
  IE = 0x82;

  while(1){
    while(TF0 == 0);
    pin = ~pin;
  }
}
```

Proteus Simulation:



Task 3:

Generate a signal of 100Hz with 40% duty cycle on P1.2 When a user presses a button at P2.3 the signal changes to 300Hz with 60% duty cycle.

- Button pressed means Press and release.

Calculation:

Frequency of signal, f1 = 100Hz

Time Period, T1 = 1/100 = 0.01s = 10ms

Duty Cycle = 40%

On-Time = 40% of 10ms = 4ms

Off-Time = 10 – **(On-Time)** = 6ms

Frequency of signal, f2 = 300Hz

Time Period,T2 = 1/300 =0.003s = 3.3ms

Duty Cycle = 60%

On-Time = 60% of 3.3ms = 1.98ms

Off-Time = 10 – **(On-Time)** = 1.32ms

So, we need to create four time delays of 4ms, 6ms, 1.98ms and 1.32ms in our program.

For 4ms:

Subtract 4000_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex} - 4000_{dec} = F05F_{\mathbf{hex}}$

For 6ms:

Subtract 6000_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex} - 6000_{dec} = E88F_{\mathbf{hex}}$

For 1.92ms:

Subtract 1920_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex}$ - 1920_{dec} = $F87F_{\mathbf{hex}}$

For 1.32ms:

Subtract 1320_{dec} from $FFFF_{hex}$

Hence, we get the starting value of the 16-bit counter, i-e

$FFFF_{hex}$ - 1320_{dec} = $FAD7_{\mathbf{hex}}$

Code:

lab_task.c

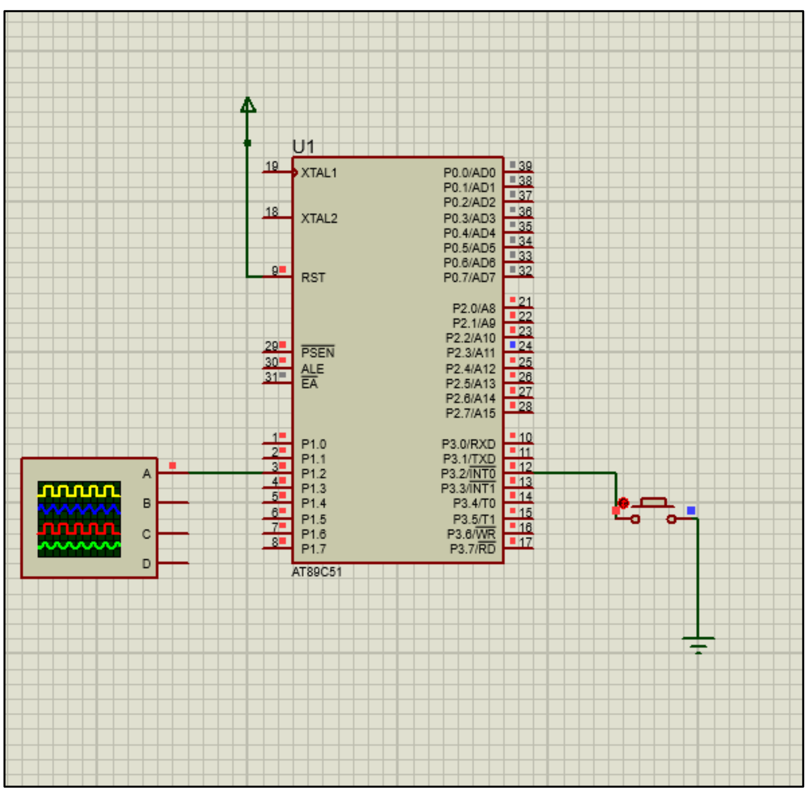
```c
#include<reg51.h>
#include<stdio.h>

sbit pin = P1^2;
sbit flag = P2^3;

sbit btn = P3^2;

void start_timer0(){
  TR0 = 1;
}

void e_i_0() interrupt 0{
  flag = ~flag;
}

void timer0() interrupt 1{

  if(flag){
      if(~pin){
        TH0 = 0xF8; //2ms delay
        TL0 = 0x2F;
      }

      else{
        TH0 = 0xFA; //1.3ms delay
        TL0 = 0xEB;
      }
```

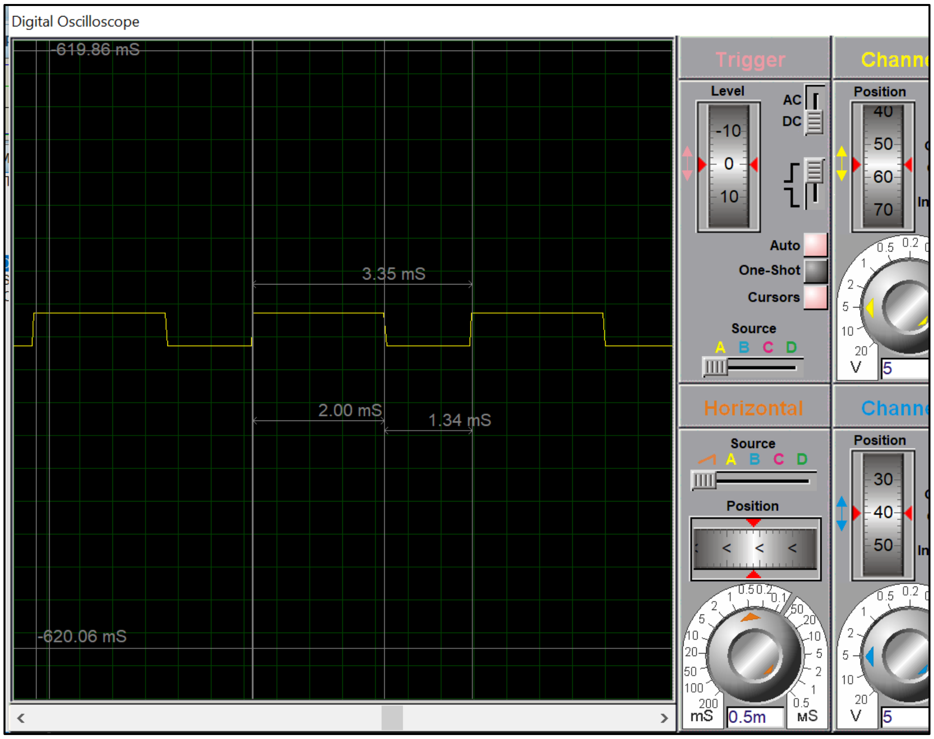
lab_task.c

```c
        }

    else{
        if(~pin){
          TH0 = 0xF0; //4ms delay
          TL0 = 0x5F;
        }

        else{
          TH0 = 0xE8; //6ms delay
          TL0 = 0x8F;
        }
      }
    }

void start_timer(){
    TMOD = 0x01;
    IE = 0x83;
}

void main(void){
        btn = 1;
        TR0 = 1;    //Part of TCON reg
        EX0 = 1;
        IT0 = 1;

        start_timer();

```

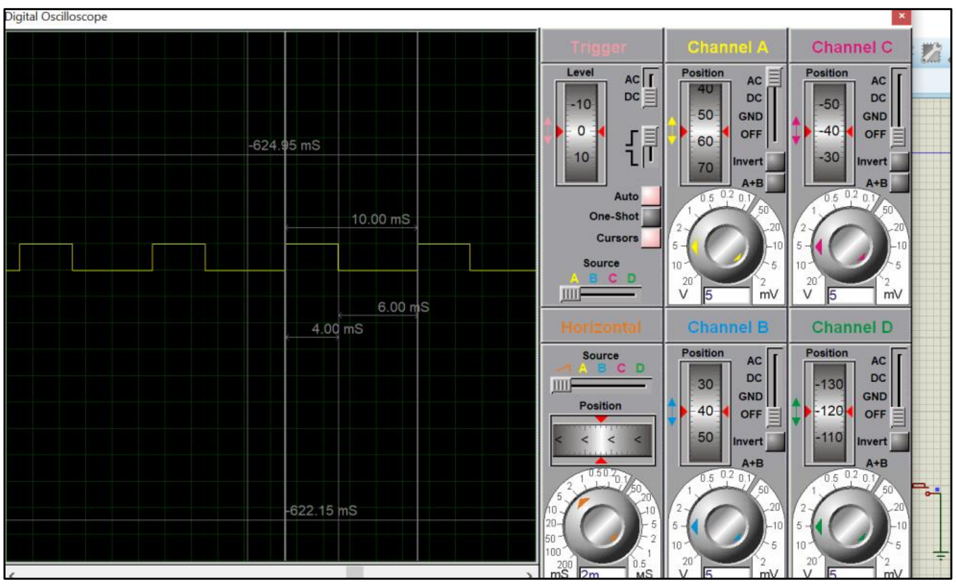
```c
}
void start_timer(){
   TMOD = 0x01;
   IE = 0x83;
}

void main(void){
         btn = 1;
         TR0 = 1;     //Part of TCON reg
         EX0 = 1;
         IT0 = 1;

         start_timer();

         while(1){
           while(TF0 == 0);
           pin = ~pin;
         }
}

```

Proteus Simulation:





Digital Oscilloscope
-619.86 mS
3.35 mS
2.00 mS
1.34 mS
-620.06 mS
Trigger
Level
Auto
One-Shot
Cursors
Source
Horizontal
Position
0.5m



Digital Oscilloscope
-624.95 mS
10.00 mS
6.00 mS
4.00 mS
-622.15 mS
Trigger
Channel A
Channel C
Horizontal
Channel B
Channel D